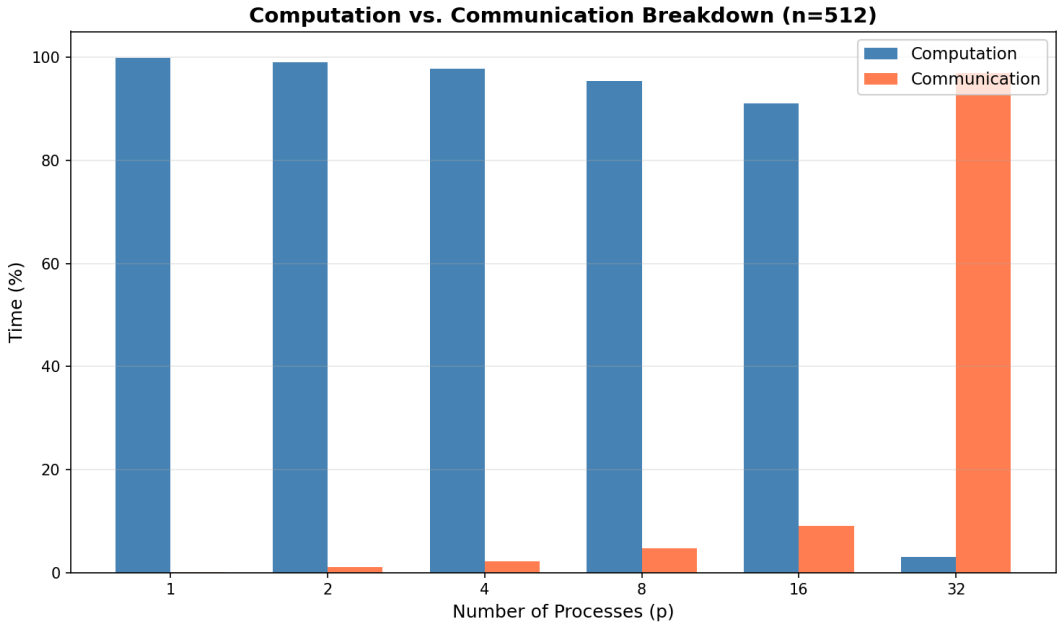
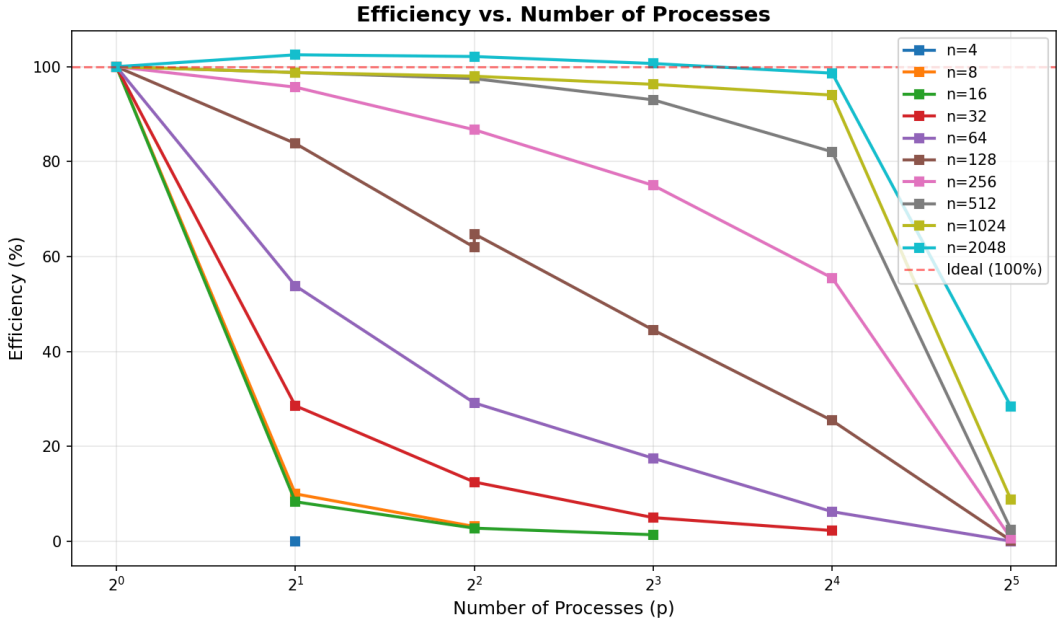
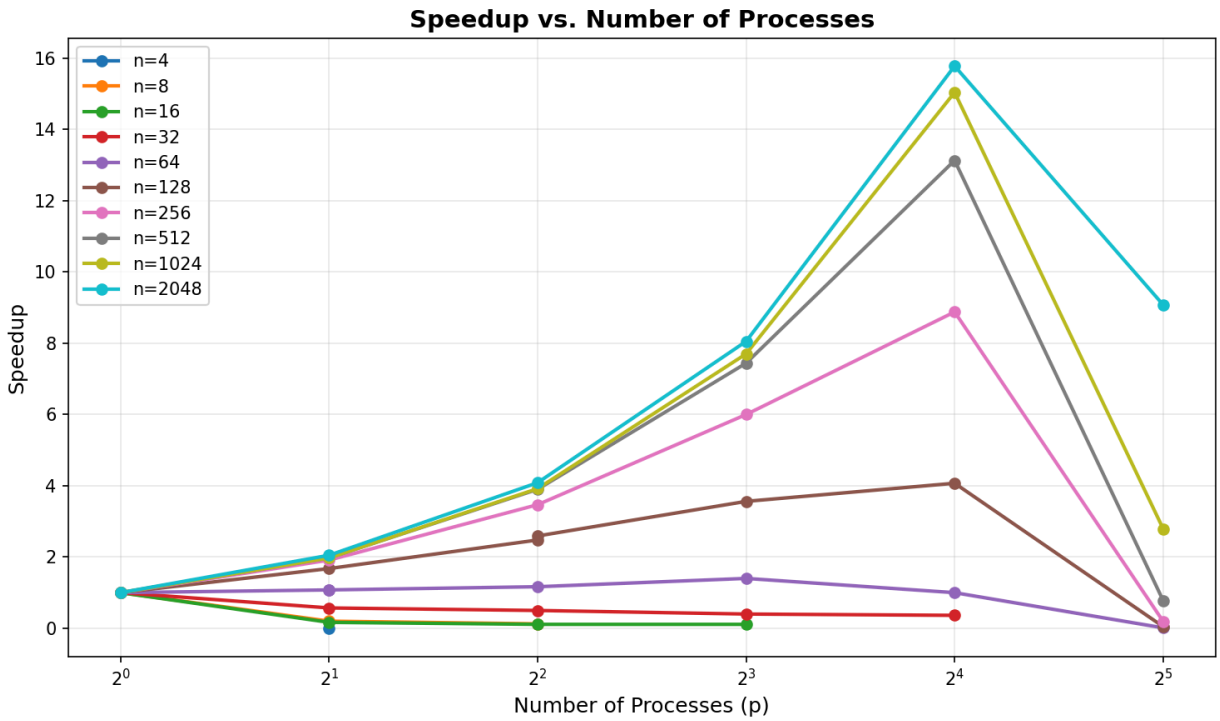
Results and Scalability Analysis

Dillon Sherling and Kyan Kotschevar-Smead


Computation vs. Communication Breakdown (n=512)
Computation
Communication
Time (%)
0
20
40
60
80
100
1
2
4
8
16
32
Number of Processes (p)


Efficiency vs. Number of Processes
n=4
n=8
n=16
n=32
n=64
n=128
n=256
n=512
n=1024
n=2048
Ideal (100%)
Efficiency (%)
0
20
40
60
80
100
2^0
2^1
2^2
2^3
2^4
2^5
Number of Processes (p)



Theoretical Scaling Behavior

The grid is decomposed using **row-wise block decomposition**, where each process owns *n/p* rows of the *n x n* board. Each process therefore updates:

$$n^2/p$$

cells per generation.

Thus the **computation cost per process** is:

$$T_{comp} = O(n^2/p)$$

Each generation also requires exchanging **two ghost rows** with neighboring processes. Since each row has nnn cells, communication cost follows the standard message model:

$$T_{comm} = O(\alpha + \beta n)$$

where α represents latency and β*n* represents message transfer time.

The total parallel time per generation can therefore be approximated as:

$T_p = O(n^2/p) + O(\alpha + \beta n)$

As *p* increases, computation decreases but communication remains roughly constant. Therefore we expect speedup to improve initially, flatten as communication becomes significant, and potentially degrade when latency dominates.

Speedup Results

The results follow the expected theoretical behavior.

For **small board sizes**, speedup decreases as the number of processes increases. This occurs because the local workload becomes extremely small while communication overhead remains constant.

For **large board sizes (e.g., n=1024n=1024n=1024 and n=2048n=2048n=2048)**, near-linear speedup is observed up to around **16 processes**. In this region computation dominates communication, allowing processors to be efficiently utilized.

Beyond approximately **16 processes**, speedup begins to decline. At this point the computation per process becomes small enough that communication latency begins to dominate runtime.

Efficiency Results

Efficiency is defined as:

$Efficiency = Speedup/p$

Large grid sizes maintain high efficiency for small and moderate numbers of processes. However, efficiency decreases as *p* increases because communication overhead becomes a larger fraction of the total runtime.

Small grids exhibit very low efficiency due to insufficient computational work per process.

Communication vs Computation

The communication/computation breakdown confirms this scaling behavior.

For small process counts, **computation dominates runtime**. As the number of processes increases, the percentage of time spent communicating increases steadily. At high process counts (e.g., $p = 32$), communication becomes the dominant cost.

This is consistent with the theoretical model where communication cost does not decrease with increasing p, while computation cost does.

Conclusion

The experimental results match the expected theoretical scaling behavior of a row-decomposed stencil computation. Large problem sizes achieve strong speedup until communication overhead begins to dominate. The scalability limit occurs when the computation per process becomes too small to amortize communication costs.